Research Progress

Toward Byzantine Fault Tolerance in Kubernetes Control Planes

Chiharu Tomimori

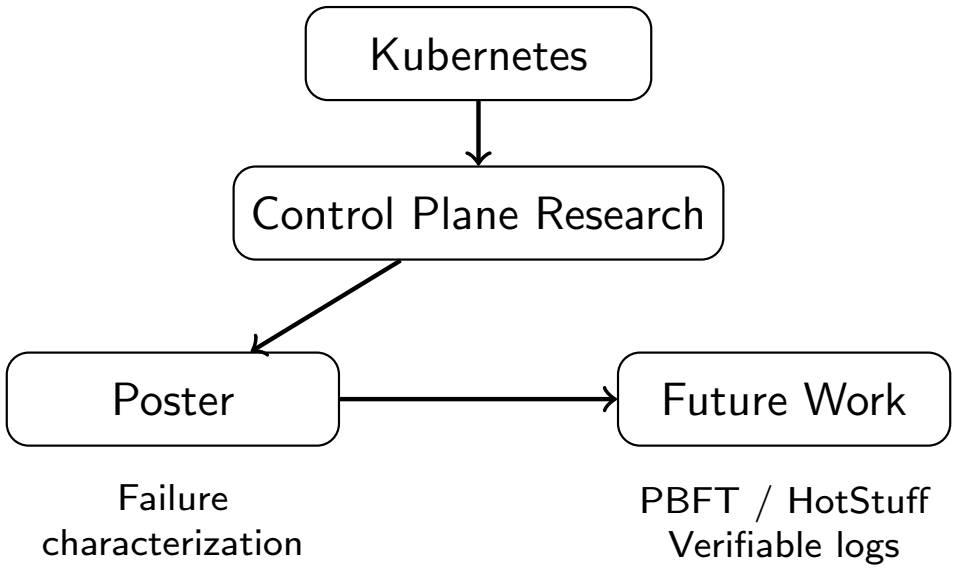
July 30, 2026

Research Direction

- Focus: Kubernetes as a distributed systems research topic
- Short-term goal: poster presentation in 5 weeks
- Long-term goal: study BFT for Kubernetes control planes

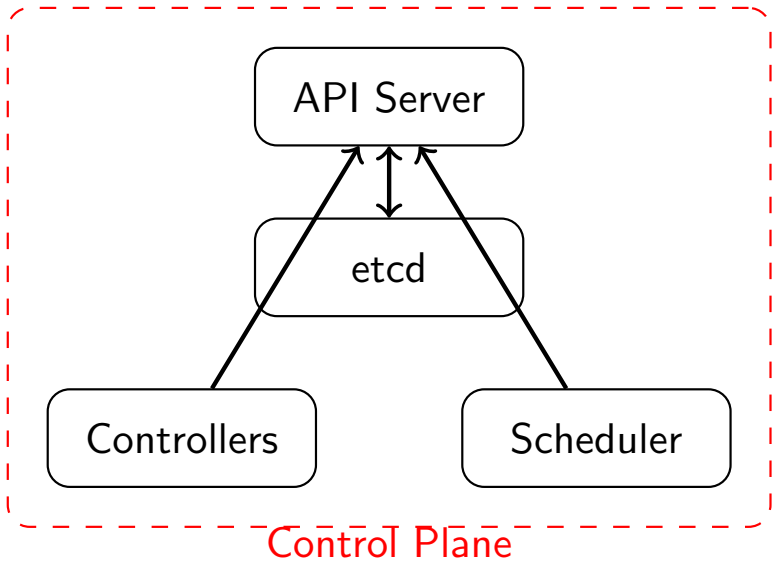
Main Research Question

How do crash failures and selective communication failures differently affect Kubernetes control-plane availability and convergence?



Why This Topic?

- Kubernetes depends on the control plane to maintain cluster state.
- etcd stores critical data such as Deployments, ConfigMaps, and scheduling-related state.
- Standard etcd uses Raft, which mainly tolerates crash failures.
- A component may remain alive but behave abnormally:
  - delay messages,
  - drop messages,
  - communicate only partially.

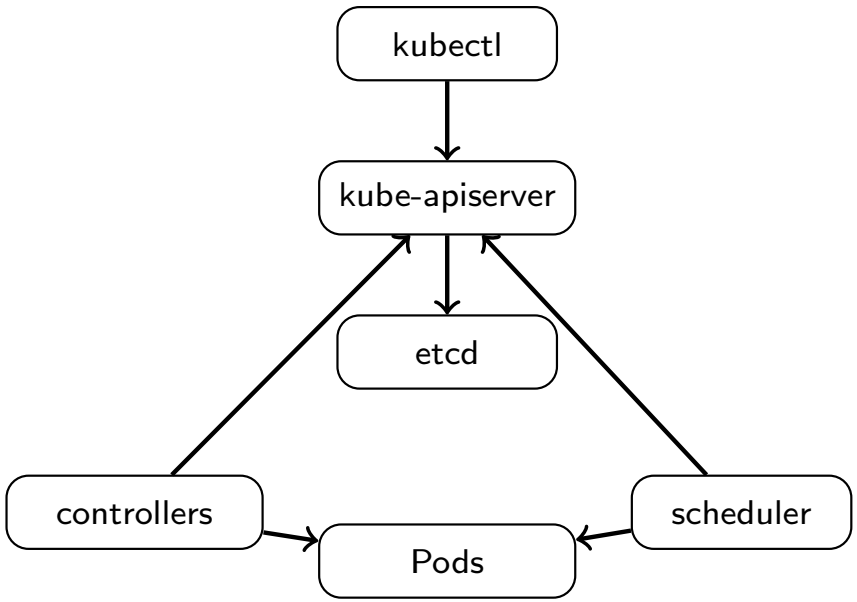


What I Learned This Week

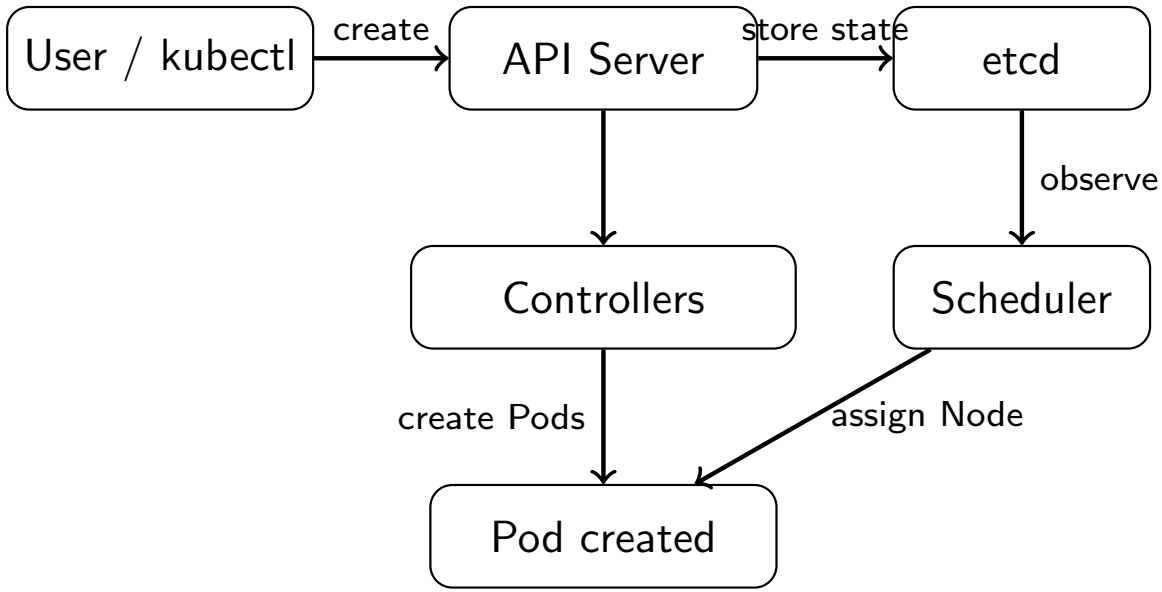
Key Understanding

Kubernetes is not directly creating Pods from `kubectl`. It is a control loop system:

- user request → API server
- desired state stored in etcd
- controllers reconcile the gap
- scheduler assigns Pods



Deployment Creation Flow



- Kubernetes stores **desired state**, not one-shot commands.
- State inconsistency in the control plane can affect convergence.

Scope of the Poster Project

Category	Included	Excluded
Target components	etcd, API server, control-plane communication	Worker nodes
Faults	follower crash, leader crash, delay, selective partition	full Byzantine response forgery
Operations	deployment creation, scaling, ConfigMap update	large-scale workloads
Goal	characterize failure impact	implement PBFT/HotStuff

Why this scope?

Five weeks is too short for full BFT integration. So I focus on what failure behaviors matter first.

Reference Paper

On Byzantine Fault Tolerance in Multi-Master Kubernetes Clusters

G. M. Diouf, H. Elbiaze, and W. Jaafar

Future Generation Computer Systems, vol. 109, pp. 407–419, 2020.

Related Work Positioning: KmMR

KmMR

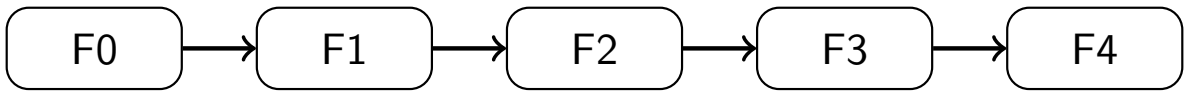
- Byzantine fault tolerance for multi-master Kubernetes
- Uses BFT-SMaRt
- Proposes a BFT-based solution

Aspect	**KmMR**	**My Work**
Goal	BFT solution	characterize failures
System	modified K8s	standard K8s
Method	BFT-SMaRt	fault injection
Output	solution perf.	problem understanding

- My work is a **preparatory step** toward future BFT integration.

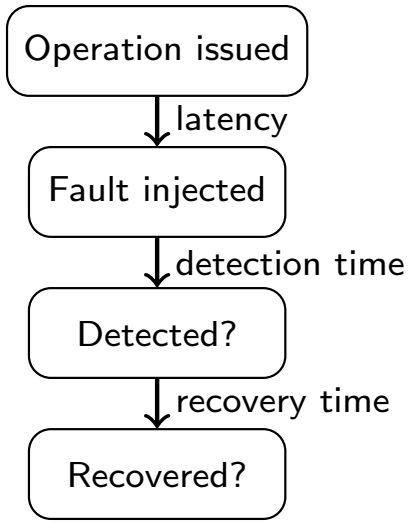
Planned Fault Scenarios

ID	Scenario	Observation
F0	Normal operation	baseline latency and convergence
F1	etcd follower crash	whether service continues correctly
F2	etcd leader crash	failover and recovery time
F3	Network delay	API latency and convergence impact
F4	Selective partition	partial communication without full stop



Evaluation Metrics

- **Success rate**: does the operation complete?
- **Latency**: how long does it take?
- **Convergence time**: when do desired Pods become ready?
- **Recovery time**: how long after a failure?
- **Detection**: does Kubernetes notice the abnormal condition?



5-Week Plan

Week	Plan
1	Fix scope, understand control plane flow, read KmMR, prepare environment
2	Baseline measurements and crash-fault experiments
3	Delay and selective communication-failure experiments
4	Analyze results and connect them to PBFT / HotStuff motivation
5	Finalize poster, figures, and presentation story

This Week's Progress

- Fixed the poster scope
- Understood the control-plane operation flow
- Started positioning the work against KmMR

Takeaways

- Kubernetes is a **desired-state distributed control system**.
- etcd is central because it stores the cluster state.
- Crash failures and communication abnormalities are not the same.
- Before introducing PBFT or HotStuff, I first need to identify what kinds of failures actually hurt Kubernetes.
- This poster project builds the foundation for future BFT-based Kubernetes research.